

Experiment # 12

Implementation of K-MEANS Clustering Algorithm

Objectives:

The objective of this lab is to

- Learn and understand the concept of K-means.
- Implement K-means Algorithm with Python.

Software Tools:

- Python IDLE or SPYDER

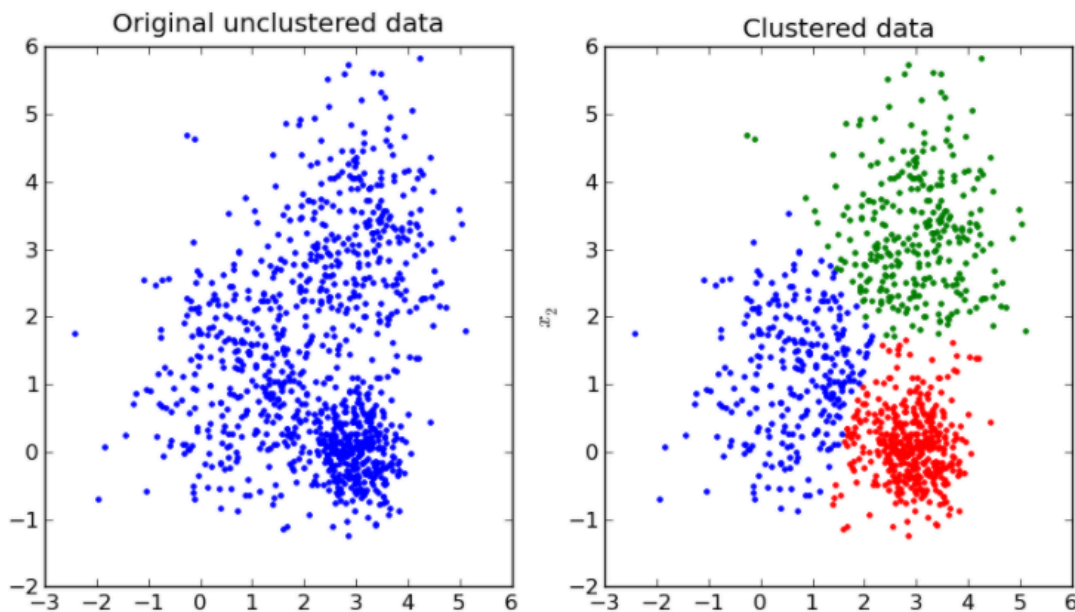
Theory:

Unsupervised Learning

Today, the majority of the machine learning world is dominated by a class of algorithms that belong to the family of **supervised learning**. What if we only have the data points but not the category information of them? **Unsupervised learning** refers to another class of algorithms that try to find the patterns in the data without any explicit labeling.

Clustering

Have you ever organized your bookshelf in a way that the books pertaining to the same subjects are in the same racks or same block? You most likely have. You already know about grouping similar objects together. While the idea is drastically simple, the amount of use cases influenced by this idea is enormous. In machine learning literature, this is often referred to as clustering - automatically grouping similar objects to the same groups. **Clustering** is a type of Unsupervised learning. This is very often used when you don't have labeled data. K-Means Clustering is one of the popular clustering algorithms. The goal of this algorithm is to find groups(clusters) in the given data.



Applications:

Web search: When you search for something with a keyword on the web, think of the amount of information the search engine processes in order to get you a result. Generally, the number of relevant web pages (also referred to as documents) with respect to a particular keyword is huge. Clustering can help to organize these web pages into meaningful groups and thereby enhancing the way the search result is presented.

Image recognition: Take the example of hand-written digits' recognition which is like the hello-world problem in the field of computer vision. There exists a wide array of deep neural network architectures which actually beat human-level performance on this task (and similar tasks like this). Now, there can be many shapes of a single digit. The way you write the digit '2' can be largely different from that of mine. It is also practically impossible to collect all these types and train a supervised learning model to recognize them. Clustering can help bypass this problem by automatically grouping these little sub-classes of separate digits and enhance the overall performance of the mother model.

2	6	8	9	3	4	7	5	6
3	4	7	9	5	5	6	7	2
5	8	7	0	9	4	3	5	4
5	2	3	4	9	5	6	7	8

K-Means Algorithm

The K Means algorithm is iterative based, it repeatedly calculates the cluster centroids, refining the values until they do not change much.

The k-means algorithm takes a dataset of ' n ' points as input, together with an integer parameter ' k ' specifying how many clusters to create (supplied by the programmer). The output is a set of ' k ' cluster centroids and a labeling of the dataset that maps each of the data points to a unique cluster.

In the beginning, the algorithm chooses k centroids in the dataset. Then it calculates the distance of each point to each centroid. Each centroid represents a cluster and the points closest to the centroid are assigned to the cluster. At the end of the first iteration, the centroid values are recalculated, usually taking the arithmetic mean of all points in the cluster.

After the new values of centroid are found, the algorithm performs the same set of steps over and over again until the differences between old centroids and the new centroids are negligible.

The implementation can be divided into the following:

- **Handle Data:** Clean the file, normalize the parameters, given numeric values to non-numeric attributes. Read data from the file and split the data for cross validation.
- **Find Initial Centroids:** Choose k centroids in random.
- **Distance Calculation:** Finding the distance between each of the datapoints with each of the centroids. This distance metric is used to find the which cluster the points belong to.
- **Re-calculating the centroids:** Find the new values for centroid.
- **Stop the iteration:** Stop the algorithm when the difference between the old and the new centroids is negligible.

Sample Example having random Data Generated K-Means Algorithm

```
import numpy as np
```

```
import pandas as pd
from matplotlib import pyplot as plt
from sklearn.datasets.samples_generator import make_blobs
from sklearn.cluster import KMeans

X, y = make_blobs(n_samples=300, n_features=2, center_box=(-10,10),
                  centers=3, cluster_std=0.60, shuffle=True,
                  random_state=0)

plt.scatter(X[:,0], X[:,1], s=30, c='cyan')
plt.show()

kmeans = KMeans(n_clusters=3, init='k-means++', max_iter=300, n_init=10, random_state=0)
pred_y = kmeans.fit_predict(X)

plt.scatter(X[:,0], X[:,1], s=30, c='blue')
plt.scatter(kmeans.cluster_centers_[0], 0],
            kmeans.cluster_centers_[1], 1], s=100, c='red')
plt.show()
```

Example

```
import cv2

import numpy as np

import matplotlib.pyplot as plt

# read the image

image = cv2.imread("C:\\Users\\Dell\\Pictures\\butterfly.png")

# convert to RGB

image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# reshape the image to a 2D array of pixels and 3 color values (RGB)
```

```
pixel_values = image.reshape((-1, 3))

# convert to float

pixel_values = np.float32(pixel_values)


print(pixel_values.shape)

# define stopping criteria

criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 100, 0.2)


# number of clusters (K)

k = 3

_, labels, (centers) = cv2.kmeans(pixel_values, k, None, criteria, 10, cv2.KMEANS_RANDOM_CENTERS)

# convert back to 8 bit values

centers = np.uint8(centers)


# flatten the labels array

labels = labels.flatten()


# convert all pixels to the color of the centroids

segmented_image = centers[labels.flatten()]

# reshape back to the original image dimension

segmented_image = segmented_image.reshape(image.shape)

# show the image

plt.imshow(segmented_image)

plt.show()

# disable only the cluster number 2 (turn the pixel into black)

masked_image = np.copy(image)
```

```
# convert to the shape of a vector of pixel values
masked_image = masked_image.reshape((-1, 3))

# color (i.e cluster) to disable
cluster = 2

masked_image[labels == cluster] = [0, 0, 0]

# convert back to original shape
masked_image = masked_image.reshape(image.shape)

# show the image
plt.imshow(masked_image)

plt.show()
```

Example

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

df = pd.DataFrame({
    'x': [12, 20, 28, 18, 29, 33, 24, 45, 45, 52, 51, 52, 55, 53, 55, 61, 64, 69, 72],
    'y': [39, 36, 30, 52, 54, 46, 55, 59, 63, 70, 66, 63, 58, 23, 14, 8, 19, 7, 24]
})

np.random.seed(200)

k = 3

# centroids[i] = [x, y]

centroids = {
    i+1: [np.random.randint(0, 80), np.random.randint(0, 80)]
    for i in range(k)
}

fig = plt.figure(figsize=(5, 5))
```

```
plt.scatter(df['x'], df['y'], color='k')

colmap = {1: 'r', 2: 'g', 3: 'b'}

for i in centroids.keys():

    plt.scatter(*centroids[i], color=colmap[i])

plt.xlim(0, 80)

plt.ylim(0, 80)

plt.show()

## Assignment Stage

def assignment(df, centroids):

    for i in centroids.keys():

        # sqrt((x1 - x2)^2 - (y1 - y2)^2)

        df['distance_from_{}'.format(i)] = (

            np.sqrt(

                (df['x'] - centroids[i][0]) ** 2

                + (df['y'] - centroids[i][1]) ** 2

            )

        )

    centroid_distance_cols = ['distance_from_{}'.format(i) for i in centroids.keys()]

    df['closest'] = df.loc[:, centroid_distance_cols].idxmin(axis=1)

    df['closest'] = df['closest'].map(lambda x: int(x.lstrip('distance_from_')))

    df['color'] = df['closest'].map(lambda x: colmap[x])

    return df

df = assignment(df, centroids)

print(df.head())

fig = plt.figure(figsize=(5, 5))
```

```
plt.scatter(df['x'], df['y'], color=df['color'], alpha=0.5, edgecolor='k')
```

```
for i in centroids.keys():
```

```
    plt.scatter(*centroids[i], color=colmap[i])
```

```
plt.xlim(0, 80)
```

```
plt.ylim(0, 80)
```

```
plt.show()
```

```
import copy
```

```
old_centroids = copy.deepcopy(centroids)
```

```
def update(k):
```

```
    for i in centroids.keys():
```

```
        centroids[i][0] = np.mean(df[df['closest'] == i]['x'])
```

```
        centroids[i][1] = np.mean(df[df['closest'] == i]['y'])
```

```
    return k
```

```
centroids = update(centroids)
```

```
fig = plt.figure(figsize=(5, 5))
```

```
ax = plt.axes()
```

```
plt.scatter(df['x'], df['y'], color=df['color'], alpha=0.5, edgecolor='k')
```

```
for i in centroids.keys():
```

```
    plt.scatter(*centroids[i], color=colmap[i])
```

```
plt.xlim(0, 80)
```

```
plt.ylim(0, 80)
```

```
for i in old_centroids.keys():
```

```
    old_x = old_centroids[i][0]
```

```
    old_y = old_centroids[i][1]
```

```
    dx = (centroids[i][0] - old_centroids[i][0]) * 0.75
```

```
    dy = (centroids[i][1] - old_centroids[i][1]) * 0.75
```



```
ax.arrow(old_x, old_y, dx, dy, head_width=2, head_length=3, fc=colmap[i], ec=colmap[i])

plt.show()

## Repeat Assignment Stage

df = assignment(df, centroids)

# Plot results

fig = plt.figure(figsize=(5, 5))

plt.scatter(df['x'], df['y'], color=df['color'], alpha=0.5, edgecolor='k')

for i in centroids.keys():

    plt.scatter(*centroids[i], color=colmap[i])

plt.xlim(0, 80)

plt.ylim(0, 80)

plt.show()

while True:

    closest_centroids = df['closest'].copy(deep=True)

    centroids = update(centroids)

    df = assignment(df, centroids)

    if closest_centroids.equals(df['closest']):

        break

fig = plt.figure(figsize=(5, 5))

plt.scatter(df['x'], df['y'], color=df['color'], alpha=0.5, edgecolor='k')

for i in centroids.keys():

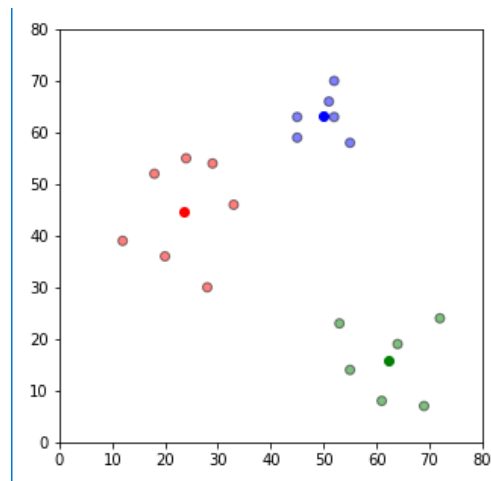
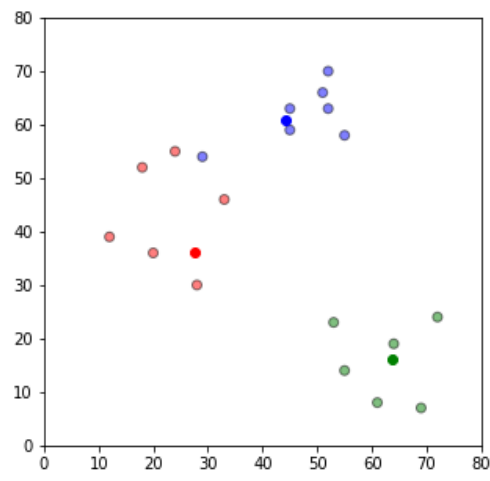
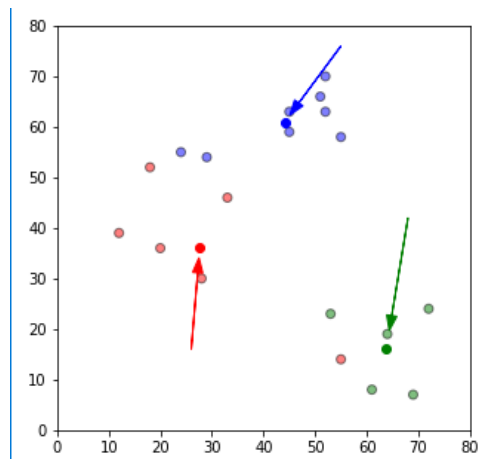
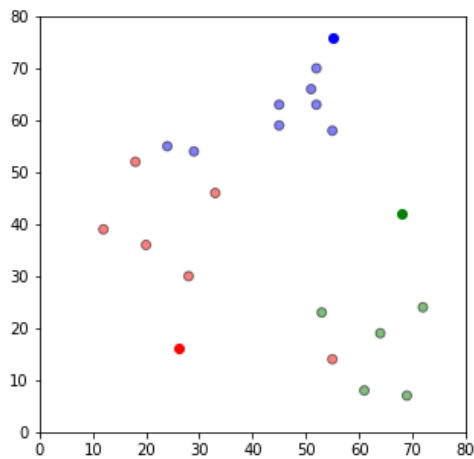
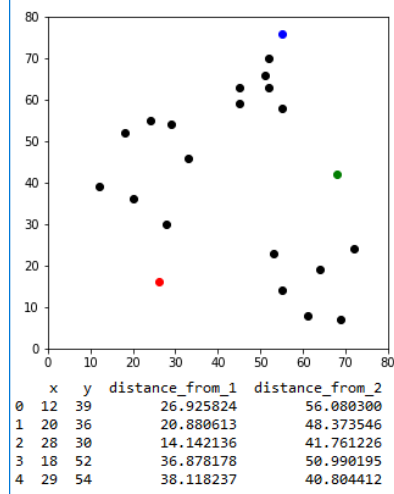
    plt.scatter(*centroids[i], color=colmap[i])

plt.xlim(0, 80)

plt.ylim(0, 80)

plt.show()
```

```
In [8]: runfile('H:/Semester Data/Semester 7/AI Lab/Lab 12/example3a.py',
            wdir='H:/Semester Data/Semester 7/AI Lab/Lab 12')
```



Task

1. **Write** a Python program to implement the K-Means algorithm with different dataset and use different Distance Formula to calculate distance between point. Also draw the proper graphs to graphically show the output of Algorithm.

NOTE: Do not use built-in K-Means function for this task. Implement K-Means Algorithm properly and show all the steps properly to define proper segments.

For help, check the following links

https://matplotlib.org/devdocs/gallery/subplots_axes_and_figures/subplots_demo.html

<http://benalexkeen.com/k-means-clustering-in-python/>